

What a world model’s uncertainty outputs actually report: an independent reproduction of the Robotic World Model

Anonymous authors

Paper under double-blind review

Abstract

We reproduce the proprioceptive dynamics model of Li, Krause and Hutter (*Robotic World Model*, arXiv:2501.10100v1) and its uncertainty-aware follow-up (arXiv:2504.16680v1) from scratch on CPU, checking it against the released reference at the gradient level first. **The base paper’s central training claim reproduces.** Autoregressive training beats teacher forcing 4.61-fold on the reference’s own relative-L1 error, on held-out episodes, under a rule committed to git before the runs that tested it. **Neither uncertainty output the follow-up adds is usable as an interval.** At the horizon its own imagination rollouts run to, the ensemble disagreement it penalises rewards with is $33.4\times$ smaller than the realised error, covering 4.61% of outcomes at $\pm 1\sigma$ against a calibrated two thirds. The per-member σ , which the method computes and discards, is worse by three orders of magnitude, and we derive why: the implemented objective’s optimum is $\sigma = 0$. **As a ranking it survives adversarial testing.** It beats the forecast step index — a free counter neither paper ran — at every horizon, and with both the rollout and the forecast depth held constant it still correlates +0.419 with realised error, so it is not merely reporting which episode is hard. **The interval is repairable.** One multiplier per horizon, fitted on one held-out episode and scored on the other, restores nominal coverage on every held-out cell; a single global multiplier manages 2 of them. No number here is typed; every claim this work has retracted on its own evidence is kept in the record (§9).

1 Introduction

A world model that reports its own uncertainty is more useful than one that does not, and the uncertainty-aware Robotic World Model reports one. This paper asks what that number means.

We came to the question sideways. Our aim was an ordinary reproduction: rebuild the proprioceptive dynamics model from scratch, check it against the released implementation, and see whether the paper’s central training claim holds. It does. But the same rebuild made a second question cheap to ask, because we had a from-scratch model, the released checkpoint, and a harness that could score both: *is the predicted σ calibrated?* Neither of the two the checkpoint emits is — the per-member σ by three to four orders of magnitude, the ensemble disagreement the method actually uses by one to two — and for the first of them the reason is structural rather than incidental.

This is a reproduction paper, and we mean the term in its stronger sense: the contribution is not that the numbers came out the same, but what systematically re-measuring the method reveals about where it is robust and where it is not. §10 collects the lessons in a form a practitioner can apply without reading the rest. Three things distinguish the work from a re-run of the authors’ code.

We rebuilt rather than imported. The forward pass, the loss and the training step are written from scratch and then checked against the reference: outputs match bitwise, and losses and gradients match to 0.000e+00 across 7 loss terms and 106 parameter tensors before any training begins (Appendix A). A discrepancy found later is therefore a property of the method, not of our wiring.

Decision rules were committed before the data. The verdicts below were fixed in advance, in git, with timestamps a reader can check (§9, Figure 4). One of them returned "cannot be settled" and we report that too.

We retract our own findings when they fail. Six numbered claims in this work are withdrawn on evidence this project produced, and two further retractions withdraw framings rather than numbers — one of them the claim that a pre-registration was pre-registered at all. All of them are kept in the record rather than deleted (§9).

Contributions.

- **A from-scratch reimplement verified at the gradient level.** Outputs match the released module bitwise; losses and gradients match to 0.000e+00 across 7 loss terms and 106 parameter tensors, before any training (Appendix A).
- **The base paper’s central training claim reproduces,** at a factor of $4.61\times$ on relative-L1 over 3 seeds, under a rule committed to git before the runs existed (§5).
- **The first calibration measurement of either uncertainty output.** Both are overconfident by one to four orders of magnitude, with intervals over independent trajectories at every horizon; and the aleatoric collapse is derived analytically from the implemented objective rather than observed (§6.2, §6.3).
- **A candidate mechanism for the epistemic failure, from source.** The five members share one trunk, one recurrent state and 89.15% of each member’s parameters, so their spread cannot express uncertainty the trunk does not already carry (§6.4).
- **The ranking claim tested against a free baseline neither original ran,** and against six controls, the last of which removes trajectory difficulty rather than forecast depth and is the only one that isolates within-rollout information (§6.7).
- **A working repair:** one multiplier per horizon, fitted on one held-out episode and scored on the other, restores nominal coverage where a global multiplier does not (§6.8).
- **Six retractions of our own numbered claims,** plus two of framings, kept in the record with the evidence that withdrew them (§9).

2 Related work

This paper measures whether an ensemble-disagreement penalty is calibrated, and separates its use as a *ranking* from its use as a *scale*. Both questions have a literature, and one paper asks almost exactly ours four years earlier. Stating that plainly is not a concession: it tells a reader which part of what follows is a new measurement and which part is a new measurement *of a new object*.

The direct precedent. Lu, Ball, Parker-Holder, Osborne and Roberts (*Revisiting Design Choices in Offline Model-Based Reinforcement Learning*, ICLR 2022) compare uncertainty heuristics in offline model-based RL under protocols built, in their words, to "capture the specific covariate shift induced by model-based RL", explicitly in order to assess calibration. They report Spearman rank and Pearson bivariate correlation against true model error **separately**, and observe that "despite the similar rank correlations ρ , the bivariate correlations r can vary considerably" — that is, a configuration can preserve ordering while changing the relationship between the penalty’s magnitude and the error’s. That is our ranking-versus-scale distinction, on the same family of penalties.

Their results also bear on ours directly, and in the same direction. They find the ensemble standard deviation — the exact quantity `system_dynamics.py:126` computes and `envs/base.py:166` applies — to correlate with model error *better* than the penalties of MOPO and MOREL, being "strikingly similar" to the latter but better behaved. Our §6.7 finds the same quantity to be a usable ranking signal. Two independent measurements, on different systems, agreeing.

So what is left for us. Lu et al. ask their question of simulated benchmarks, of models they train themselves, and they do not ask whether the penalty is a calibrated interval — rank and bivariate correlation are both ordering-and-shape statistics, not coverage. We ask it of a **released checkpoint** from a pipeline its authors deployed on hardware, where the interval reading is consequential; we measure **coverage** against a nominal, not only correlation; and we do it with a from-scratch reimplement verified against the reference at the gradient level, so that a discrepancy is a property of the method rather than of our wiring. The contribution is not the idea of checking. It is the object checked, the quantity measured, and the standard of verification.

Where the parameterisation comes from, and why it matters more than one repository. The bounded $\log\sigma$ head that §6.3 shows has its optimum at $\sigma = 0$ is not this codebase’s invention. It is inherited, line for line, from the probabilistic ensembles of Chua, Calandra, McAllister and Levine (PETS, NeurIPS 2018). PETS’s Appendix A.1 gives

```
logvar = max_logvar - softplus(max_logvar - logvar)
logvar = min_logvar + softplus(logvar - min_logvar)
```

and `architectures/mlp.py:92-93` is those two lines in log-standard-deviation rather than log-variance, with `system_dynamics.py:302` supplying PETS’s regulariser on the bounds.

What is **not** inherited is the objective. PETS uses "the negative log prediction probability as our loss function", and it is that likelihood’s $\log\sigma$ term which opposes $\sigma \rightarrow 0$. `system_dynamics.py:283` substitutes squared error on a reparameterised sample, which has no such term. So the collapse §6.3 derives follows from the *substitution*, not from the parameterisation — and that makes §6.3 larger than one repository: **any descendant of this lineage that replaced the likelihood with a sampled squared error inherits the same optimum**. We state that as a hypothesis and mark it clearly: we have not tested any other descendant, and testing one is out of scope here (§12).

The method being reproduced sits in a well-populated family. MOPO (Yu, Thomas, Yu, Ermon, Zou, Levine, Finn and Ma, NeurIPS 2020) penalises the reward by an ensemble uncertainty estimate to solve a pessimistic MDP; MOREL (Kidambi, Rajeswaran, Netrapalli and Joachims, NeurIPS 2020) builds an unknown-state detector from pairwise ensemble disagreement instead; and both branch short model rollouts from real states in the manner of MBPO (Janner, Fu, Zhang and Levine, NeurIPS 2019). The follow-up we reproduce adapts MOPO’s penalty into MBPO’s loop, which is what "MOPO-PPO" names. The rollout-length-versus-model-error trade MBPO introduces is the one the follow-up’s 100-step imagination horizon sits inside (§6.2, and X-13 in the ledger).

What "ensemble" is supposed to mean. Deep ensembles (Lakshminarayanan, Pritzel and Blundell, NeurIPS 2017) are several networks trained from *different random initialisations and different data orderings*, and their spread is the uncertainty estimate. That definition is the reference point for §6.4: the released checkpoint’s five members share one GRU trunk, one recurrent hidden state, and 89.15% of each member’s state-prediction parameters. Whatever that spread measures, it is not what a deep ensemble measures, and §6.4 sets out what follows.

Three sources of uncertainty, and the one nobody here estimates. Abbas, Sokota, Talvitie and White (ICML 2020) separate predictive uncertainty in model-based RL into aleatoric noise, parameter uncertainty, and **model inadequacy**, and observe that selective-planning work attends almost entirely to the second. The checkpoint we measure emits an aleatoric term and a parameter-uncertainty term, discards the first before use (§6.1), and has no estimate of the third at all. Model inadequacy is precisely the component that compounds with rollout depth, which is the shape §6.9 reports: σ flat while error grows.

Why miscalibration is the expected finding rather than a surprising one. Modern networks are systematically miscalibrated (Guo, Pleiss, Sun and Weinberger, ICML 2017), and calibration degrades further under covariate shift, worsening with distance from the training distribution (Ovadia, Fertig, Ren, Nado, Sculley, Nowozin, Dillon, Lakshminarayanan and Snoek, NeurIPS 2019). An autoregressive rollout manufactures its own covariate shift, increasing with depth, so horizon-dependent calibration failure is the shape one should expect. Our contribution on this axis is the *magnitude* and the *mechanism*, not the direction. Finally, §6.8’s per-horizon multiplier is a coarse instance of calibrated regression (Kuleshov,

Fenner and Ermon, ICML 2018): a post-hoc map fitted on held-out data. We present it as an application of that idea to a horizon index, not as a new one.

Every entry above was checked against the paper itself — title, full author list, venue and year from the arXiv record, and for any sentence we attribute, the sentence matched verbatim against the paper’s own text. 10 of 10 entries verified, 9 of 9 attributed fragments verbatim (results/t1_bibliography_verified.json). No entry was added that was not verified.

3 Setup

Data. The released dataset is 10,000 rows of ANYmal D proprioceptive state and policy actions at 50 Hz. It is not one recording: it is ten concatenated 20-second episodes, and its termination column is identically zero, so nothing in the file marks the boundaries. The reference window builder therefore marks all 9,961 windows valid, including 352 that splice one episode’s end onto the next one’s start. The usable, episode-respecting count is 9,609 — 10,000 rows, less 39 that cannot start a full window, less 352 that cross a boundary. The contamination rate is 3.53%.

Model. A GRU-based ensemble predicting the next proprioceptive state, with a mean head and a bounded log- σ head, plus auxiliary heads for contact and termination. The paper describes two loss terms; the implementation has 7.

Evaluation. Two arenas, held separate throughout: *out-of-sample*, the two episodes withheld from training, and *in-sample*, the eight used for it. We report both, because the released evaluation draws its trajectories from training data and the distinction is invisible in the original.

Effective sample size. Trajectory count is not sample size. Two 400-step trajectories whose spans overlap are not independent evidence, and the out-of-sample arena contains only 4 mutually non-overlapping 400-step trajectories. Every interval in this paper is a bootstrap over independent trajectories, and every table reports that count.

3.1 Metrics

Every metric below is stated as implemented, with the `file:line` of the implementation; each citation is read back and checked against its own source text on every build (20 of them). This section exists because two metrics in this project once disagreed in *direction* at $h = 1$, and because a choice between two aggregations of the same metric once inverted a comparison against the released model. A reader who cannot see the denominator cannot check the headline.

Relative-L1 is the reference’s own metric, reproduced verbatim in behaviour (`model_training.py:203`) so that our numbers are comparable to the upstream’s printed one. On config-normalised states, per forecast step,

$$r_t = \frac{\sum_{d=1}^{45} |\hat{s}_{t,d} - s_{t,d}|}{\sum_{d=1}^{45} |s_{t,d}|}$$

and the reported figure is the flat mean over trajectories and steps,

$$e = \frac{1}{B(T - t_0)} \sum_{b=1}^B \sum_{t=t_0+1}^T r_t^{(b)}$$

with $t_0 = \text{history_horizon} = 32$: the first 32 steps are teacher-forced and excluded. The denominator is recomputed at every step and is a 45-term sum in normalised space, so it can pass through zero — which is why this metric goes non-finite on low-dimensional state groups, and why a second one exists.

Normalised RMSE fixes the denominator once, over the training episodes only:

$$\text{nRMSE}_t = \frac{\sqrt{\frac{1}{45} \sum_{d=1}^{45} \text{MSE}_{t,d}}}{\frac{1}{45} \sum_{d=1}^{45} \sigma_d^{\text{tr}}} \quad \text{with} \quad \text{MSE}_{t,d} = \frac{1}{B} \sum_{b=1}^B (\hat{s}_{t,d}^{(b)} - s_{t,d}^{(b)})^2$$

where the scale constant is

$$\sigma_d^{\text{tr}} = \text{sd}(\{\tilde{s}_{i,d} : \text{episode}(i) \in \mathcal{E}_{\text{train}}\})$$

computed once, stored in `results/step4_0a_results.json`, never recomputed per step and never derived from held-out data. A value of 1.0 means no better than predicting the training mean. **The aggregation matters and is form 1:** pool the per-dimension mean squared errors, then divide — a ratio of means. The alternative, a mean of per-dimension ratios, gives whichever dimension has the smallest scale unbounded leverage, and the choice between the two once inverted a published-model comparison in this project (§9). Form 2 is reported only for continuity with figures published before that was found.

Coverage at $\pm k\sigma$ is the fraction of scalar (trajectory, forecast step, state dimension) triples whose absolute realised error falls within k times the σ predicted for that same triple:

$$\text{cov}_{\pm k\sigma}(h) = \frac{1}{B h 45} \sum_{b=1}^B \sum_{t=t_0+1}^{t_0+h} \sum_{d=1}^{45} \mathbf{1} \left[\frac{|\hat{s}_{t,d}^{(b)} - s_{t,d}^{(b)}|}{\sigma_{t,d}^{(b)}} \leq k \right]$$

Three things a reader needs and the prose did not previously give. It is pooled over all three axes with equal weight per triple. It is **cumulative** over steps 1..h — coverage "at h" averages the whole rollout up to h and is not the value at step h, and the same convention governs every horizon-indexed quantity in this paper. And because the statistic is built from an *absolute* error, $z \leq k$ is the two-sided event, so the calibrated targets are $\text{erf}(k/\sqrt{2})$: **68.27%** at $\pm 1\sigma$ and **95.45%** at $\pm 2\sigma$.

The overconfidence factor is how many times larger the typical realised error is than the typical predicted σ :

$$\rho(h) = \frac{\text{mean}_{b,t \leq h,d} |\hat{s}_{t,d}^{(b)} - s_{t,d}^{(b)}|}{\text{mean}_{b,t \leq h,d} \sigma_{t,d}^{(b)}}$$

It too is a **ratio of means**, not a mean of ratios — the latter is unbounded whenever a single σ approaches zero, which is exactly the regime §6.3 puts these models in. One caution on reading it: $\rho = 1$ is *not* calibration. A calibrated Gaussian has $\text{mean}|\text{error}| / \sigma = \sqrt{2/\pi} = 0.7979$. ρ is reported as a magnitude of miscalibration and coverage is the calibrated reading, which is why both appear everywhere.

Which metric each headline uses. The A/B training claim (§5) is relative-L1, because the claim is about reproducing the upstream’s comparison and that is the upstream’s metric. The calibration claims (§6.2) are the overconfidence factor and coverage, because neither error metric involves σ at all. The ranking claims (§6.7) are Pearson correlations between the applied scalar penalty and total absolute error, because a ranking claim is about order rather than scale. Every headline number in the abstract names its metric.

Horizons. Curves are reported at $h \in \{1, 8, 32, 100, 128, 368\}$. Two of those are load-bearing and the rest are landmarks. **h = 100** is the method’s own imagination rollout length — the horizon over which the uncertainty-penalised policy loop actually runs this model (arXiv:2504.16680 Table S9 in v1, Table S11 in v3; the value is unchanged between them, and v3 states it in prose as well). **h = 368** is the upstream’s open-loop diagnostic length: `len_eval_trajectory` = 400 minus the 32-step teacher-forced prefix, which is the curve the follow-up plots as its uncertainty figure. It is $3.68\times$ the method’s own rollout length and it is not a deployment horizon; earlier drafts of this paper called it one, and that label is withdrawn. Both are kept in every table, because $h = 368$ is what makes our numbers comparable to the original’s *figure* while $h = 100$ makes them comparable to the original’s *method*.

4 What the original papers claim, and which claims we test

A reproduction that does not say what it left alone invites the reader to assume it tested everything. It did not.

We tested four claims and left eight untested. The four are the base paper’s autoregressive -versus-teacher-forcing comparison and its claim that teacher forcing generalises poorly (§5), and the follow-up’s two claims about what its uncertainty outputs report (§6). The eight we did not test are, without exception, claims about **policy learning or hardware**: zero-shot transfer, the sample-efficiency result, the comparisons against SHAC and Dreamer, generality across robot morphologies, and the core claim that penalising rewards by disagreement improves the learned policy. Each needs a simulator, an RL loop and an ANYmal; this work trains no policy at all. §12 states what that bounds, and Appendix E sets out what testing them would take.

For all 4 of the claims we did test, the original reports no quantitative figure. Each is asserted qualitatively and shown in a plot; none is given a number in text, caption or table. So our $4.61\times$ is neither a confirmation of a published figure nor a contradiction of one — it is the first figure attached to the claim, and the same is true of the follow-up’s “strong correlation” between disagreement and error, for which §6.7 supplies the first coefficient. Where a magnitude is legible only from a plotted curve we say so rather than estimating it from the axis.

Appendix F gives the full table, claim by claim, with what the original states, where it states it, and our verdict.

5 The base paper’s central claim reproduces

Claim under test. Training the dynamics model on its own autoregressive rollouts beats training it with teacher forcing, at the horizons the method deploys at.

Rule, committed in advance (commit `efc35b8`, and it names conditions rather than outcomes). Three conditions, all required: the out-of-sample gap at $h = 368$ excludes zero under a bootstrap over independent trajectories; the sign is consistent across episodes; and the effect survives at 10,000 iterations rather than only at the paper’s 2,500.

Result. Every condition holds. We give the evidence in order of how little it depends on the small held-out sample.

The sign test, which does not depend on n . At $h = 368$ the per-episode gap favours autoregressive training on **10 of 10** episodes — an exact two-sided binomial test, $p = \mathbf{0.0020}$. This is one test on ten paired episodes, it uses no bootstrap, and no multiplicity correction touches it. Unlike the per-dimension counts in §6, episodes are genuinely separable units, so a binomial null is admissible here.

Its scope needs stating plainly, because the arena labels invite a stronger reading than it supports. 8 of the 10 episodes are training data for *both* arms. The test is therefore a valid **paired** comparison — the two arms saw identical data, so any episode-level difference is attributable to the training rule and not to what either model had memorised — but it is not ten out-of-sample episodes, and it does not measure generalisation. The out-of-sample effect size below carries that burden, on 4 independent trajectories.

The in-sample arena, where the sample is larger. The same comparison on the eight training episodes has 16 independent 400-step trajectories against the held-out arena’s 4 — $4\times$ more — and gives the same direction at every horizon and checkpoint.

The out-of-sample effect size, over 3 seeds. Autoregressive training reaches $\mathbf{0.3582 \pm 0.0283}$ against teacher forcing’s $\mathbf{1.6497 \pm 0.2858}$ (standard deviation over seeds, `ddof=1`) — a factor of $\mathbf{4.61\times}$.

Seed spread is not symmetric between the arms and that is worth stating: Arm A ranges 0.3341–0.3894 across seeds (7.9% relative), Arm B 1.4241–1.9710 (17.3%). Teacher forcing is more than twice as variable across seeds as autoregressive training at this horizon, so a single-seed comparison of these two arms is

unreliable in a way a reader should know about. An earlier draft of this paper quoted the single-seed figures 0.3509 and 1.5540; those came from the seed that happened to be favourable to Arm A and unfavourable to Arm B, and the three-seed ratio is $4.61\times$ rather than $4.4\times$.

For a single seed the bootstrap over trajectories gives 95% interval $[0.56, 2.05]$ on $n = 4$ independent trajectories. **That interval should not be read as an ordinary one:** four trajectories admit 256 distinct resamples, so any bootstrap tail is quantised to steps of 0.39%, and the interval is coarse by construction. It is offered as corroboration of the sign test, not as the primary evidence.

Against a baseline, because neither number means anything without one. The hold-last floor — predicting that nothing changes — scores **0.9930** in the same cell. Autoregressive training beats it by **$2.8\times$** . **Teacher forcing is $1.56\times$ worse than assuming nothing changes at all**, which is the sharper statement of what exposure bias costs here: the arm that reaches a lower training loss ends up predicting the future worse than a model that makes no prediction.

What does not hold, and we say so. At $h = 8$ — the horizon the model is trained on — the same comparison out-of-sample gives a gap of 0.008 whose interval includes zero. The advantage is a long-horizon phenomenon. An earlier rule of ours, anchored at $h = 8$, returned "cannot be settled"; anchoring a rule to the horizon the claim is actually about was a correction we had to make in advance of the runs, not after them (§9).

The pattern is consistent across the design. Under the correct cluster bootstrap, the out-of-sample gap excludes zero in **4 of 4** long-horizon cells — both trajectory lengths crossed with both checkpoints — and in **0 of 4** at $h = 8$. These figures are relative-L1; the nRMSE aggregation is reported separately and does not change the direction.

Multiplicity. Those 4 cells sit in a family of 8 out-of-sample comparisons, so we state the correction rather than leaving it to a reader. All 4 of 4 still exclude zero at a Bonferroni level of $0.05/8$, and Holm–Bonferroni rejects **4 of 4**. The sign test above is unaffected either way.

5.1 The data budget, which is the one part of the sample-efficiency claim we can measure

The base paper’s headline is a sample-efficiency result: policies transfer to hardware from 6,000,000 state transitions of world-model pretraining against $\sim 250\text{M}$ for the model-free baseline (Table I). We cannot test it — it is a claim about policy learning and hardware. But its *world-model* half is a claim about a quantity we can count exactly, and ours is directly comparable.

Our arms consume 7,991 distinct state transitions. That is the 7,999 rows of the eight training episodes less one per episode boundary (8 of them), a transition being a consecutive pair of rows inside one episode. It is deliberately not the 7,687 training windows, which overlap almost completely — consecutive 33-step windows start one row apart — nor the 640,000 window draws a run makes, which resample the same data with replacement. Against the reference’s 6,000,000, that is **$751\times$ less data, 0.133% of its world-model budget**.

A dynamics model trained on 0.133% of the reference’s data still reproduces the autoregressive-versus-teacher-forcing result at $4.61\times$ and still beats the hold-last floor by $2.8\times$ at $h=368$. That is what this paper can contribute to the sample-efficiency question without training a policy.

Three limits, in the same breath. It is not a reproduction of the 6,000,000-against-250M comparison, which is about policy learning and which we do not touch. It says nothing about whether a policy trained inside our model would transfer to hardware, or anywhere. And our model is evaluated on the same narrow distribution it trained on — one robot, one gait, one terrain, velocity commands from a single bounded box — where the reference’s 6,000,000 transitions span considerably more. A smaller data budget buys less than it appears to when the evaluation distribution shrinks with it.

6 Neither of the checkpoint’s uncertainty outputs is usable as an interval

6.1 Which quantity the method actually uses

The released checkpoint emits **two** uncertainty quantities, and the method consumes only one of them. This has to be settled before any calibration number means anything.

`system_dynamics.py:125` computes an **aleatoric** term, the mean over ensemble members of each member’s predicted σ . `system_dynamics.py:126` computes an **epistemic** term, the standard deviation across the members’ mean predictions. In `envs/base.py:142` the aleatoric term is bound to a local variable that is never read again; the epistemic term is stored, returned to the policy loop at `:158`, and applied at `:166` as a reward penalty with weight -1.0 .

The paper agrees with its code. arXiv:2504.16680v1 Eq. 4 defines the penalised quantity as — the numbering is unchanged in the current v3 — $u = \text{Var}_b[\mu_b]$, the variance across ensemble members, and Eq. 5 applies it as $\tilde{r} = r - \lambda u$. The per-member predicted variance enters the training objective and nothing downstream.

Eq. 4 specifies a **variance** while `system_dynamics.py:126` computes a **standard deviation**, which with $\lambda = 1$ differ by a square. We asked, and the first author confirms the code is operative: the penalty is applied to the standard deviation as intended, and Eq. 4 is "more of a high-level explanation" (personal communication, 21 August 2026; the exchange is reproduced in full, anonymised, in the supplementary material as `SUPPLEMENTARY_CORRESPONDENCE.md`, so these quotations are checkable rather than asserted). We measure the code’s quantity throughout, which is now known to be the intended one.

The same correspondence confirms the discard directly: "the aleatoric term is not used in downstream training. It is reported in Fig. 3 (right) as an analysis of the model behavior." So what follows is not an implementation slip being reported back to its authors — it is the intended design, and the aleatoric head exists to shape training and be inspected rather than to be consumed.

So the aleatoric head — the one the state loss and the bound loss shape, and the one §6.3 explains — is computed on every imagination step and discarded. We report both quantities below. Our own arms are ensemble size 1, where the epistemic term is identically zero by construction, so the epistemic measurement is possible only on the released checkpoint.

What the follow-up does and does not claim, stated before we measure anything. It does not claim its uncertainty is a calibrated interval. The follow-up’s §5.1 claims the epistemic term "closely follows the trend of the prediction error" and that this "justifies its role as a trust metric", and of the aleatoric term it observes only that it "remains low, reflecting small stochasticity in the environment". Our measurement **supports the first claim** — the epistemic ordering is real and strong. What follows is therefore not a refutation of a calibration claim nobody made. It is three things the papers do not address: that the aleatoric head is discarded before use, that neither quantity is usable as a scale, and that the low aleatoric value has a different cause than the one offered.

6.2 The measurement

For each model we compute the mean predicted σ , the mean absolute realised error, and the fraction of realised errors falling inside $\pm 1\sigma$. A calibrated Gaussian puts 68.3% inside $\pm 1\sigma$.

model	mean error / mean σ , whole 368-step rollout	coverage at $\pm 1\sigma$, h=1	coverage at $\pm 1\sigma$, h=100
faithful Arm A (sampled MSE)	$52.2 \times [39.6, 69.7]$	11.67% [7.96, 15.19]	2.17% [1.59, 2.75]
corrected Arm A (<code>gaussian_nll</code>)	$10.9 \times [7.8, 14.7]$	42.78% [24.44, 62.04]	9.8% [6.80, 12.81]
teacher-forced Arm B	$315 \times [177, 509]$	12.96% [7.04, 20.93]	1.22% [0.84, 1.61]
released checkpoint	$7,878 \times [5,410, 9,934]$	0.56% [0.00, 1.67]	0.08% [0.06, 0.11]

Every cell carries a 95% interval from a cluster bootstrap over whole trajectories, $n_{\text{independent}} = 4$; where three seeds contribute, seeds are pooled inside each draw rather than resampled, because seeds are

not trajectories (§9). At $n_independent = 4$ the bootstrap has 256 distinct resamples and the intervals are quantised at that resolution.

All four rows are the held-out arena — the two episodes withheld from our own arms, $n_independent = 4$ — because that is the only arena on which our arms can be scored fairly. On the aleatoric head every model is overconfident by between one and four orders of magnitude (Figure 1); that is the quantity §6.1 shows the method discards.

A note on the released checkpoint's row, so the next table does not read as a contradiction. Its $7,878\times$ is measured on those same 4 trajectories, for comparability with the three arms beside it. The released checkpoint trained on all ten episodes, so its own best-sampled figure is the $11,683\times$ below, at $n_independent = 20$. Both are correct; they are different arenas, and neither is a held-out measurement of the released checkpoint, which has no held-out arena in this dataset.

The quantity the method does use is also uncalibrated. On the released 5-member checkpoint over all 10 episodes, $n_independent = 20$ non-overlapping 400-step trajectories. We use all ten rather than the held-out pair here because the released checkpoint trained on all ten, so restricting it to two buys no independence and costs four fifths of the sample — the same argument this paper makes about that checkpoint elsewhere. The held-out-only version at $n_independent = 4$ is in the supplementary material (`results/task_b2_epistemic.json`). The epistemic column agrees in direction with this one at all 5 of 5 horizons; the aleatoric column agrees at 4 of 5 — it flips sign at $h=8$, where both readings sit close enough to chance that the sign is not meaningful in either. Where the two tables differ materially we say so.

h	aleatoric err/ σ [95% CI]	aleatoric $\pm 1\sigma$	epistemic err/ σ [95% CI]	epistemic $\pm 1\sigma$ [95% CI]	epistemic $\pm 2\sigma$	dims $r>0$	permutation P
1	$1,827\times$ [915, 3,110]	0.11%	8.3 $\times$ [6.1, 9.7]	16.22% [13.44, 18.89]	30.11%	44/45	0.0060
8	$3,034\times$ [1,638, 4,849]	0.08%	$15.1\times$ [10.3, 19.2]	9.99% [8.60, 11.32]	19.76%	45/45	0.0085
32	$4,525\times$ [2,492, 7,196]	0.07%	$22.6\times$ [15.5, 28.7]	6.95% [5.99, 7.85]	13.68%	45/45	0.0355
100	$11,683\times$ [7,970, 15,846]	0.04%	33.4 $\times$ [28.7, 39.0]	4.61% [4.12, 5.09]	9.27%	45/45	—
128	$14,934\times$ [10,564, 19,700]	0.03%	$34.2\times$ [30.6, 38.2]	4.37% [3.96, 4.81]	8.75%	45/45	0.3725
368	$20,669\times$ [15,666, 25,688]	0.02%	$34.4\times$ [29.8, 40.3]	3.59% [3.27, 3.92]	7.19%	45/45	0.0823

Epistemic is $600\times$ better than aleatoric and still wrong by **8.3** $\times$ at one step and **33.4** $\times$ [**28.7, 39.0**] at $h = 100$, the method's own imagination rollout length, with $\pm 1\sigma$ coverage of 4.61% [4.12, 5.09] where a calibrated Gaussian gives 68.27%. At the open-loop diagnostic horizon of $h = 368$ it is $34.4\times$ [29.8, 40.3] with 3.59% coverage — barely different, which is why the re-anchoring changes the reading and not the conclusion. **Total** uncertainty, $\sqrt{\text{aleatoric}^2 + \text{epistemic}^2}$, equals the epistemic value to four significant figures at every horizon, because the aleatoric term is too small to move it.

The larger sample changes one thing materially, and it is a correction to our own earlier reading. At $n_independent = 4$ the epistemic ordering looked like chance at short horizon — 23 of 45 dimensions at $h=1$ — and we had described it as a long-horizon effect. At $n_independent = 20$ it is 44 of 45 at $h=1$, with mean $r = +0.662$, the *strongest* mean correlation of any horizon. The in-sample permutation test says the same (§6.6). The short-horizon "chance" result was an artifact of four trajectories, not a property of the model, and we record it as such rather than keeping the more interesting-sounding horizon story.

The released checkpoint is no longer the only ensemble measured. Three Arm A arms at ensemble size 5 (§6.7, 3 seeds, out-of-sample, $n_independent = 4$) give, averaged over seeds:

h	epistemic err/ σ [95% CI]	$\pm 1\sigma$ [95% CI]	$\pm 2\sigma$	dims $r>0$
1	$2.1\times$ [1.7, 2.5]	35.93% [29.63, 42.22]	60.37%	35.0/45
8	$6.3\times$ [4.4, 7.5]	14.10% [11.02, 17.18]	26.62%	37.3/45
32	$8.0\times$ [5.9, 9.8]	10.79% [8.53, 13.71]	20.50%	34.3/45
100	10.5 $\times$ [9.0, 11.5]	8.19% [6.80, 9.59]	16.11%	44.0/45
128	$11.0\times$ [9.5, 11.9]	7.79% [6.81, 8.77]	15.34%	44.0/45
368	13.0 $\times$ [11.9, 13.9]	6.30% [5.84, 6.77]	12.51%	40.7/45

Our arms are **better calibrated than the released checkpoint and fail the same way**: $10.5\times$ overconfident at $h = 100$ against its $33.4\times$, with 8.19% coverage where a calibrated Gaussian gives 68.27%. §6.4

establishes that the two are the same architecture in the respect that matters here, so this is a comparison of like with like. Being an order of magnitude closer to calibrated is not being calibrated.

The last column of the table above gives permutation P-values over whole trajectories, not binomial ones, computed on the same 20 trajectories as the counts beside them; §6.6 explains why a binomial null is inadmissible here and how far it was wrong. These are five tests on one family and none survives Holm–Bonferroni across the arena’s 25 cells — the smallest is faithful (mse) $h=368$ at 0.0042 against a threshold of 0.002. Read the column as a consistency check on direction, not as five independent findings.

The scalar penalty as actually applied — `means.std(0).sum(-1)` at `envs/base.py:166` — correlates **+0.605** with total absolute error over the rollout, 95% CI $[+0.545, +0.694]$ from a bootstrap over whole trajectories, $n_{\text{independent}} = 20$ (7,360 pooled trajectory-step points). An earlier draft quoted this correlation with neither an interval nor an n . The interval resamples whole trajectories, not trajectory-step pairs, which would narrow it by about the square root of the rollout length.

6.3 Why the aleatoric head collapses: the optimum is $\sigma = 0$

This subsection explains the aleatoric column and only that column. Ensemble disagreement is not shaped by the mechanism below, and why *it* is miscalibrated is not established here.

It also supplies the alternative explanation promised in §6.1. The follow-up reads the low aleatoric value as reflecting "small stochasticity in the environment". The observation is correct and the reading is not: σ is low because $\sigma = 0$ is the optimum of the loss that trains it, and it would be low on any dataset, stochastic or not.

The state loss is squared error on a *sample* drawn from the predicted Gaussian, not a likelihood:

$$\mathcal{L} = \mathbb{E}[(\mu + \sigma\varepsilon - y)^2] = (\mu - y)^2 + \sigma^2$$

which is minimised at $\sigma = 0$ for any μ . There is no $\log\sigma$ term to oppose it. The bound term that appears to oppose it does not, because `max_logstd` is not an independent parameter — it is constructed as `min_logstd + exp(log_delta_logstd)`, so

$$\overline{\log \sigma_{\max}} - \overline{\log \sigma_{\min}} = \overline{\exp(\log \Delta_{\log \sigma})}$$

and `min_logstd` cancels algebraically, taking no gradient from that term. The floor the interval closes onto therefore freezes while the interval closes: a one-way ratchet.

We predicted the collapse from this algebra before training, then observed it. Across all 24 runs the collapse is linear in iteration count and its rate is nearly identical (Figure 3a). Rates are fitted on 18 of those runs: the 6 10,000-iteration runs are excluded from the rate statistics because they continue seeds already counted at 2,500 and would double-weight them. Figure 3(a) shows all 24 runs; Figure 3(b) plots only the 18 the rate is fitted on, so the scatter and the quoted statistic describe the same set.

The 24 runs, so a reader can count them:

arm	iterations	ensemble	objective	dataset	seeds	seed ids
Arm A	2,500	1	gaussian_nll	clean	3	0, 1, 2
Arm A	2,500	1	mse	clean	3	0, 1, 2
Arm A	2,500	1	mse	contaminated	3	0, 1, 2
Arm A	2,500	1	mse	duplicated	3	0, 1, 2
Arm A	2,500	5	mse	clean	3	0, 1, 2
Arm A	10,000	1	mse	clean	3	0, 1, 2
Arm B	2,500	1	mse	clean	3	0, 1, 2
Arm B	10,000	1	mse	clean	3	0, 1, 2

Two different things are being explained here, and §6.6 separates them. *Magnitude collapse is objective-driven.* It occurs in all 15 sampled-MSE runs at a rate of $-9.3894\text{e-}05$ per iteration with a standard deviation of $6.3\text{e-}07$ — **including the teacher-forced arm**, which shares the objective — and reverses to $+3.2332\text{e-}05$ in the 3 runs that change it. *Input-independence is not.* That varies by a factor of 15.6 between two arms trained under the same objective, so the objective cannot be what produces it.

Under the corrected objective the sign flips (Figure 3b) — which is the strongest evidence that the mechanism is the objective and not the optimiser, the data or the architecture.

6.4 Why the epistemic term may be miscalibrated: the members are not independent models

§6.3 explains the aleatoric column and says of the epistemic one that the mechanism is not established. This subsection supplies a candidate, structurally symmetric to §6.3’s, from source and from the checkpoint’s own tensors. Nothing here is trained and nothing is inferred from a measurement.

The released five-member ensemble is not five models. `system_dynamics.py:34` builds **one** `state_base`. `system_dynamics.py:35-41` replicates the `heads` `ensemble_size` times, and only the heads. In the forward pass `system_dynamics.py:87` evaluates the trunk **once** and `:90` hands the identical feature vector to every head; `system_dynamics.py:126` then computes the epistemic term as the standard deviation across those heads.

The parameter counts make the scale of the sharing concrete. The state pathway is a 636,672-parameter two-layer GRU trunk plus 5 heads of 77,492 parameters each. Per member, 636,672 of 714,164 parameters — **89.15%** — are numerically identical to every other member’s. Only 10.85% differ. Across the whole released object, the two shared trunks are 63.81% of 1,995,569 parameters.

The sharing is stronger than the parameter count suggests, and this is the part that matters. The trunk owns a *single* recurrent hidden state (`rnn.py:40`), and an autoregressive rollout feeds the ensemble **mean** back into it (`system_dynamics.py:115`; `src/rwm_model.py:223` in our reimplementation). So the 5 members do not roll out independently at all. There is 1 hidden-state trajectory, and disagreement at step t is the spread of 5 two-layer MLPs read off one 256-dimensional vector at the point that single trajectory has reached.

That is the argument. **Members which share a feature extractor have correlated errors by construction, and their spread cannot express uncertainty the shared trunk does not already carry.** Where a deep ensemble in the sense of Lakshminarayanan et al. varies initialisation *and* data ordering across whole models, here only the output heads differ — so the quantity the method penalises rewards with is a lower bound on epistemic uncertainty by construction, not by accident. It is the same shape of finding as §6.3: not a training failure, a structural one.

This applies to our own arms identically, which is why §6.2’s comparison is fair. Our ensemble-5 arms build one trunk the same way (`src/rwm_model.py:164-167`), evaluate it once (`:182`), hand the same vector to every head (`:185`) and compute the same spread (`:200`). Their tensor names and parameter counts match the released checkpoint exactly — 636,672 shared, 77,492 per head, 1,995,569 in total, on all 3 arms checked. So §6.2’s “our arms fail the same way at $10.5\times$ ” compares two instances of one architecture, not two architectures. 20 source citations support the paragraphs above and each is read back from the pinned upstream and checked on every build (`results/v1_ensemble_topology.json`).

What this is and is not. It is a *candidate* mechanism, established structurally. It is not yet a demonstration that trunk-sharing is *the* explanation for the miscalibration in §6.2 — architecture could be a minor contributor to a failure dominated by something else. Establishing that needs a comparison against an ensemble which shares nothing, which is what M-44 pre-registers and what §6.10 reports. We keep the topology and the mechanism separate on purpose: the topology is a fact about the released artifact, and the mechanism is a hypothesis about that fact.

6.5 The correction fails differently rather than succeeding

The reference contains an unused `gaussian_nll` branch. Running it reverses the collapse and improves the magnitude from $52.2\times$ to $10.9\times$ overconfident. It does not produce a usable estimate, and it destroys something the faithful arm had: the σ -versus-error ordering falls from 39/45 dimensions positively correlated to 21/45, which is chance. Under the trajectory permutation test of §6.6 those counts give $P = 0.0417$ and 1.0000 out of sample, 0.0232 and 0.8734 in sample. The faithful arm’s ordering is the one result in this family that points the same way in both arenas; it is also the weakest effect of the three, and it does not survive multiplicity correction either.

6.6 The failure is one of magnitude; the ordering is weaker than it looks

Measuring the teacher-forced arm — which we had trained for §5, and which our own first three calibration tables omitted — sharpens the finding:

model	σ variation across inputs (CoV)	dims with $r(\sigma, \text{error}) > 0$, out-of-sample	perm P, out-of-sample	perm P, in-sample
faithful Arm A	0.0076	39/45	0.0417	0.0232
corrected Arm A	0.0059	21/45	1.0000	0.8734
teacher-forced Arm B	0.1188	45/45	0.2609	0.5656
released checkpoint	0.0177	20/45	0.6957	0.9738

The CoV column is the aleatoric σ in every row, which is the only σ the ensemble-size-1 arms have. Our ensemble-5 arms have both: their aleatoric CoV is comparable to the other arms’, and their *epistemic* term is far more input-dependent than any aleatoric head here, at 0.379–0.395 against the released checkpoint’s 0.0177 (§6.7). **The count column is the out-of-sample arena** ($n_{\text{independent}} = 4$), so that all four models are compared on trajectories none of our own arms was trained on. It is not the only arena, and for the released checkpoint’s aleatoric head it is not the most informative one: at $n_{\text{independent}} = 20$ over all ten episodes that head is 0/45 — negatively correlated with error on *every* dimension — against 20/45 here. §13 quotes the larger arena and says so.

Arm B’s σ is $15.6\times$ more input-dependent than the faithful arm’s, and it has the largest mean correlation of the four ($r = 0.257$). It is still $315\times$ overconfident.

The P column above is not a binomial one, and an earlier draft of this paper was wrong to make it one. Converting a count of positive per-dimension correlations to a P-value against a fair-coin null assumes the 45 state dimensions are independent trials. They are not. Position, velocity and torque for the same joint are physically coupled, and base linear and angular velocity are coupled through the gait. More importantly, error grows with rollout depth in every trajectory, so *any* σ that also grows with depth correlates with *any* trajectory’s error — including one it was never paired with.

We therefore permute whole trajectories. The null pairs each trajectory’s σ with a different trajectory’s realised error, which leaves both marginal distributions and the entire cross-dimension dependence structure intact and destroys only the association under test. The correction is large, and it is largest exactly where we leaned hardest. The worst-affected cell is teacher-forced armB at $h=368$, in the in-sample arena. It moves from $5.68e-14$ to 0.5656 — a factor of about 10^{13} — because under a null that preserves the dependence, a random re-pairing already yields 43.8 of 45 dimensions positive on average. Observing 45 of 45 against that null is close to unremarkable. A fair coin, by contrast, centres the count at 22.5 of 45; the dependence-preserving null centres it between 5.4 and 43.8 depending on model, horizon and arena.

So σ *collapsing in magnitude* is objective-driven, and σ *becoming input-independent* is not. The teacher-forced arm collapses in magnitude exactly like the autoregressive ones — same objective, same rate — while retaining $15.6\times$ more variation across inputs. Input-independence is a property of the autoregressive arms and the released checkpoint, and input-dependence and correct ranking are both achievable without the interval becoming meaningful.

One candidate mechanism, stated as a hypothesis and not a result: autoregressive feedback narrows the input distribution toward the model’s own manifold, leaving a heteroscedastic head less variation to key on. We have not tested it.

The same pattern holds for the quantity the method uses, and this is where the correction bites hardest. At $h=128$ and $h=368$ the epistemic term correlates positively with realised error on **45 of 45** dimensions, matching the best aleatoric head here on the sign count, while being $39.7\times$ overconfident. **All figures in this paragraph are the held-out arena ($n_independent = 4$),** so that the epistemic term and the four aleatoric heads are compared on identical trajectories; §6.2 quotes $34.4\times$ for the same ratio at $n_independent = 20$, which is the figure the abstract and §13 use. It does not beat Arm B's head on strength either: its mean correlation at $h=368$ is $+0.151$ against 0.257 . The two quantities rank comparably; neither is close to an interval. Under the permutation null that count gives $P = 0.0435$ out of sample and 0.0758 in sample, against $5.68e-14$ from the independent-trials test we should not have used. It still fails the horizon test the same way: σ grows $1.59\times$ from $h=1$ to $h=368$ while error grows $13.33\times$.

The horizon story we first told was backwards, and the larger arenas agree with each other against the smallest. At $n_independent = 4$ out of sample, the epistemic ordering looked strongest at long horizon (0.0417 at $h=128$, 0.0435 at $h=368$) and unremarkable at short (0.4348 at $h=1$). Both larger arenas invert that. In sample ($n_independent = 16$): 0.0060 at $h=1$, 0.0084 at $h=8$, against 0.3804 at $h=128$. Over all ten episodes ($n_independent = 20$): 0.0060 , 0.0085 and 0.3725 . Two independent arenas at four and five times the sample say the effect is strongest at *short* horizon.

The null means explain why, and the explanation is the same one that motivates §6.7. At long horizon the shared forecast-depth trend lifts the null to 41.6 of 45 , so a count of 45 is close to what chance alone delivers; at short horizon the null sits near 15.4 and the same count is genuinely surprising. The out-of-sample arena is not wrong so much as blind: at 4 trajectories its smallest attainable P-value is 0.04167 , so it cannot distinguish a strong effect from a marginal one at any horizon. We report the small arena's numbers alongside because it is the only arena that is out-of-sample for our own arms, not because it is the better measurement.

Nothing here survives multiplicity correction, in any of the three arenas. Holm–Bonferroni over each arena's 25 model $\times$ horizon cells at $\alpha = 0.05$ rejects 0 out of sample, 0 in sample and 0 over all ten episodes. Out of sample that is a property of the design rather than of the models: with 4 independent trajectories the smallest attainable P-value is 0.04167 , which already exceeds the smallest Holm threshold 0.002 , so no effect of any size could have been rejected there. In sample the miss is real — the smallest P in the family is teacher-forced armB $h=128$ at 0.0027 against a threshold of 0.002 .

So the honest form of this section's claim is narrower than the one we first wrote. **The magnitude failure is established and large; the ordering is directionally consistent across every model and horizon we measured, and is not established at conventional significance once the dependence between dimensions is respected.**

The failure is specifically magnitude calibration, in both components.

6.7 Ensemble disagreement beats the trivial baseline

The follow-up justifies ensemble disagreement as a trust metric on the grounds that it "closely follows the trend of the prediction error". Section 6.6 shows the per-dimension version of that claim is weaker than it looks. This section asks a different and, for a practitioner, more important question: **does disagreement beat something free?**

Error in an autoregressive rollout grows with depth. So the trivial competitor to any trust metric is the forecast step index — a counter. It needs no ensemble, no second forward pass and no model. If a counter ranks error as well as disagreement does, the ensemble is not earning its cost. Neither paper runs this comparison, so we do.

All three correlations below are on the scalar quantity the method actually applies — `means.std(0).sum(-1)` at `envs/base.py:166` — against total absolute error, over $n_independent = 20$ trajectories, with 95% intervals from a bootstrap over whole trajectories.

h	r(step index, error)	r(disagreement, error)	partial r(disagreement, error · index)	paired difference, disagreement − index
1	—	+0.994 [+0.918, +0.999]	—	—
8	+0.181 [+0.108, +0.349]	+0.738 [+0.537, +0.807]	+0.757 [+0.540, +0.822]	+0.556 [+0.241, +0.679]
32	+0.174 [+0.131, +0.336]	+0.735 [+0.577, +0.823]	+0.756 [+0.604, +0.841]	+0.561 [+0.298, +0.674]
128	+0.526 [+0.421, +0.643]	+0.671 [+0.604, +0.846]	+0.617 [+0.534, +0.812]	+0.145 [+0.011, +0.324]
368	+0.269 [+0.106, +0.431]	+0.605 [+0.545, +0.694]	+0.596 [+0.526, +0.687]	+0.337 [+0.147, +0.544]

($h=1$ has a single forecast step, so the index is constant and its correlation — and therefore the partial and the difference — undefined. The epistemic correlation is not, and it is the largest anywhere in this work: at one step, ensemble disagreement is very nearly a perfect ranking of realised error.)

Disagreement wins at every horizon tested. The counter reaches +0.269 over the full rollout against disagreement’s +0.605, and the index leads in 0 of 4 horizons.

The last column answers the first question — does disagreement beat the counter — and it is not the test a reader might expect. Comparing the two marginal intervals for overlap is the wrong comparison here: both correlations are measured on the *same* trajectories, so their sampling errors move together and the marginal intervals are needlessly conservative. The paired difference — resampling whole trajectories and recomputing *both* correlations inside each draw — is the appropriate test and the more powerful one. It excludes zero at **4 of 4** horizons.

The distinction matters at exactly one place. At $h=128$ the marginal intervals *do* overlap — it is the horizon where the counter is strongest (+0.526) and the margin narrowest — and an earlier draft of this paper wrongly asserted that they never do. The paired difference there is +0.145 [+0.011, +0.324], which excludes zero, but only just: +0.011 is the smallest lower bound in the table and we would not rest anything on that horizon alone.

The third column answers a different question: is disagreement merely re-encoding the clock? Partialling the step index out of both variables *lowers* disagreement’s correlation by 0.010, from +0.605 to +0.596. Almost none of what disagreement knows is explained by knowing how deep into the rollout you are. It is carrying real information about *this* rollout, not a re-encoding of the clock.

A linear control is not much of a control, so we tested four harder ones. Error does not grow linearly with rollout depth, and a control that under-fits the index leaves index-driven variance in the residual and flatters disagreement. Partialling out $\log(1 + \text{index})$ gives +0.589 [+0.512, +0.688]; a cubic in the index gives +0.582 [+0.495, +0.676]; a rank partial correlation, which removes *any* monotone dependence on depth rather than an assumed functional form, gives +0.906 [+0.856, +0.921].

The decisive one needs no model of the index-error relationship at all. Computing the correlation **within each forecast step** — across trajectories, with depth held exactly constant, so the index cannot contribute by construction — and averaging over steps gives **+0.739 [+0.711, +0.852]**, positive at 368 of 368 forecast steps with a median of +0.737.^[stepcount] The weakest figure across all 5 controls is +0.582. Disagreement is not re-encoding the clock: at a fixed depth it still knows which rollouts are going wrong.

^[stepcount]: Adjacent forecast steps on the same 20 trajectories are heavily dependent — structurally the same problem §6.6 spends a page correcting for the 45 coupled state dimensions. The count is descriptive; the interval [+0.711, +0.852] is the statistic, and no P-value attaches to 368/368.

All five of those controls remove forecast depth. None removes trajectory difficulty, and the sixth does. This is the one control in this section that was pre-registered before the statistic was computed (M-45, §9), and adding it changed how we read the five above.

Per-episode difficulty in this dataset spans 0.562 to 1.591 and is uncorrelated with commanded speed, so the units being correlated differ a great deal in *level*. If harder trajectories simply have both larger error and larger disagreement, the pooled correlation would look exactly as it does with disagreement carrying no within-rollout information whatever. Two things suggested that was worth checking: the 20-point $h = 1$ figure of +0.994, which is not the shape of a genuine per-step signal, and the within-step control coming out *above* the pooled figure, which is the signature of a between-unit effect.

Splitting the pooled correlation settles which effect carries it:

what is removed	correlation	95% CI	what it answers
nothing (pooled)	+0.605	[+0.545, +0.695]	—
everything within a trajectory — the 20 trajectory means alone	+0.878	[+0.817, +0.956]	do harder rollouts disagree more?
forecast-step means only	+0.589	—	the existing within-step control, pooled
trajectory means only	+0.470	[+0.389, +0.604]	—
both, additively (r_dd)	+0.419	[+0.318, +0.576]	at a given depth, in a given rollout, does disagreement know?

The between-trajectory correlation is **+0.878**, and the two components contribute 51.7% and 48.3% of the pooled covariance. So a large part of what §6.7 has been reporting is a between-rollout effect. **That reinterprets the fifth control rather than merely adding to it.** The within-step statistic correlates *across trajectories* at each fixed step and averages — which removes depth completely and removes trajectory difficulty not at all. It is a mean of 368 between-trajectory correlations, which is why it reads +0.739 against the pooled +0.605 rather than below it. It was described in an earlier draft as "the decisive one"; it is not, and we withdraw that description.

The decisive statistic is the double-demeaned one, and it survives. Removing the trajectory mean *and* the step mean from both variables and correlating the residuals gives **+0.419 [+0.318, +0.576]**, on $n_independent = 20$ with a cluster bootstrap over whole trajectories. M-45's threshold, fixed before the statistic was computed, was that this interval exclude zero; the rule's minimum detectable effect at this sample size is

$$\underline{\underline{r_dd}}$$

true effect of 0.086 (not detected) and 0.172 (detected). The observed effect is comfortably above it. **M-45 returns SUPPORTED:** with both the rollout and the depth held constant, disagreement still tracks error. It is not merely reporting which episode is hard.

Two qualifications a reader should carry away with that. The within-rollout effect is **materially smaller than the pooled figure** — +0.419 against +0.605 — so a practitioner should expect disagreement to separate *rollouts* better than it separates *moments within a rollout*. And it is **not established at short horizon:** r_dd's interval excludes zero at $h=100$, $h=128$, $h=368$ and spans zero at $h=8$ and $h=32$, where too few steps exist to demean against. That inverts the shape one might expect and we report it as measured.

The $h = 1$ figure survives the same test, but it is not what it looked like. At $h = 1$ the panel has one column, so +0.994 is a correlation over 20 *trajectory-level* points and nothing within a rollout is being tested at all. We checked whether trajectory difficulty manufactures it: disagreement correlates +0.006 with commanded speed and +0.040 with per-episode difficulty, and partialling both out of the disagreement–error correlation leaves +0.995 — it does not move. So the figure is real and is not a difficulty artifact. It is nevertheless a statement about **ranking whole rollouts at one step ahead**, on 20 points, and §10 now says that rather than calling it a ranking of realised error without qualification.

Does it hold on a model we trained? Everything above is measured on the released checkpoint, because our main arms run at ensemble size 1 where the epistemic term is identically zero. We therefore trained three Arm A arms at **ensemble size 5**, identical in every other setting, under a rule committed to git before the runs existed (§9, M-43). The rule asked for two things: that disagreement lead the index at every horizon, and that the paired difference exclude zero at a majority of them.

It returns DOES NOT GENERALISE. The first condition passes completely — disagreement leads the index in **12 of 12** seed-horizon cells, every paired estimate positive, +0.204 to +0.545. The second fails: the paired difference excludes zero at 1 of 4 horizons, not a majority. We report the verdict the rule returns and do not rewrite the rule.

What separates the two conditions is sample size, and we measured that rather than asserting it. Our own arms can only be scored out-of-sample on the held-out pair, $n_independent = 4$, where §6.7's own finding used 20. Subsampling four trajectories at a time from a twenty-trajectory pool, the rule's criterion fires on 75% of draws on average and on only 24% at $h=8$. That estimate is an **upper bound**, because the pool it subsamples is in-sample for these arms, where the effect is +0.425 to +0.790 against +0.204 to +0.545 on the held-out pair. So the rule was under-powered at the sample size it faced, decisively at one horizon — and we do not claim it could not have passed, only that it was committed without anyone

checking what it could detect. That is a failure of ours, and it is the same one the ledger already records as M-24: a rule anchored without regard to the regime it would be applied in.

Reported as a companion and not as a discharge: on all ten episodes ($n_{\text{independent}} = 20$, **in-sample** for these arms, which trained on eight of them) the same measurement excludes zero at 4 of 4 horizons and would have satisfied both conditions. It cannot discharge M-43, which is stated over the out-of-sample arena, and we record it only so the comparison with the released checkpoint's 20 is like for like.

We ran the baseline test expecting it to go the other way. A counter matching disagreement would have been the more consequential result — it would make the trust metric close to vacuous, since a counter is free — and that is the outcome this test was set up to expose. We record the expectation as an expectation only: it was not committed to git before the data existed, so by this paper's own standard (§9) it is not a pre-registration, and it carries none of the weight one would. It did not go that way. **On this axis the follow-up's claim survives adversarial testing against a real baseline**, and that is the strongest form of support this paper offers any claim of either original work. It coexists with §6.6 without contradiction: the *scalar* the method applies tracks error well, while the *per-dimension* sign counts we had leaned on carry far less evidence than an independent-trials test suggested. The quantity is a usable ranking signal and is still not an interval.

6.8 One constant scalar does not fix it, but a per-horizon one does

If σ had the right shape and the wrong scale, a single multiplier would repair it, and the finding would be a units problem with a one-line remedy. We tested that. A scalar was fitted on **one** held-out episode and evaluated on the **other**, in both directions, so it is never fitted on its own test set.

Fitting at one step works at one step and nowhere else. On the epistemic term — the quantity the method uses — a scalar of 5.08–5.82 brings $h=1$ coverage to 63–74%, essentially calibrated against the 68.3% target, and leaves $h=368$ at 17–21%. On the aleatoric term a scalar of 593–611 gives 64–70% at $h=1$ and 11–15% at $h=368$. Fitting over the whole rollout instead drives one-step coverage to 100% — an interval wide enough to be vacuous where the model is accurate — while still falling short at the far end.

The reason is §6.9's mechanism: a constant multiplier cannot track an error that grows while σ does not. So "right shape, wrong scale" is the charitable reading of these tables, and for a *constant* scale it does not survive.

A per-horizon scalar does work, and this is the one concrete remedy in this paper. Fitting one multiplier per horizon on one held-out episode and evaluating on the other, in both directions, so no multiplier is ever scored on the episode that produced it. The two held-out episodes contribute 4 non-overlapping 400-step trajectories between them, so each direction fits on $n_{\text{independent}} = 2$ and is scored on the other 2:

quantity	held-out cells within 10 points of 68.27%, per-horizon c	same, constant c	range of fitted c
aleatoric	12 / 12	2 / 12	592.6 – 7782 (13.1×)
epistemic	12 / 12	2 / 12	5.082 – 47.33 (9.31×)

Every held-out cell lands within 10 points of the 68.27% target for both quantities. The largest deviation over all 24 held-out cells is aleatoric at $h=100$, fitted on episode 8 and scored on the other, at 77.48% — 9.21 points off target. The two largest deviations are both at $h=100$ on the aleatoric term, in opposite directions (77.48% and 76.55%), which is a mild sign that the fitted multiplier is least stable at that horizon. The constant scalar manages 2 of 12, and those are the $h=1$ cells it was fitted at.

Three cautions a reader should apply. The per-horizon scalar has one free parameter per horizon against the constant one's one, so it *must* fit better in sample — only the held-out column above is evidence, and that is the column reported. **And the held-out column is thinner than its count suggests:** the 12 cells are 6 horizons $\times$ two fold directions on the same 4 trajectories, and each multiplier is fitted on $n_{\text{independent}} = 2$ and scored on the other 2. They are not 12 independent successes and no P-value attaches to the count; it is reported so a reader can see how thin the evidence is, alongside a result we believe. And the correction is

a calibration patch, not a fix: it leaves the model’s σ carrying no more information than before and simply rescales it by how far ahead you are looking. It is nevertheless enough to make the interval mean what it says, which is what a downstream user needs, and it costs one lookup table.

So the accurate form of this section is: **a constant scalar does not repair the interval; a per-horizon one does, and transfers across episodes.**

6.9 The structural excuse does not survive

One could argue that a model trained on an 8-step horizon cannot be expected to report calibrated uncertainty about step 368. It cannot report it about step 8 either. Inside the trained horizon, σ is flat while error grows (Figure 2):

model	σ growth, step 1 $\rightarrow$ 8	error growth, step 1 $\rightarrow$ 8
faithful Arm A	0.9241 $\times$	3.49 $\times$
corrected Arm A	1.0003 $\times$	3.41 $\times$
teacher-forced Arm B	1.0096 $\times$	6.11 $\times$
released checkpoint	1.0007 $\times$	1.79 $\times$

The faithful arm’s σ *declines* (0.9241 $\times$) while its error grows 3.49 $\times$. The coverage collapse in Figure 1(b) is therefore driven entirely by growing error against a fixed σ .

7 Defects in the released pipeline

7.1 Ten unmarked episode boundaries. §3. The window builder reads a termination column that is identically zero, so it marks all 9,961 windows valid.

7.2 Training and evaluation disagree on action alignment, and evaluation is the broken one. Row t holds the action that *produced* state t . The training path pairs states and actions index-for-index, which is causally correct. The evaluation path feeds the action from $t-1$ to predict state t — stale by one step. Scored correctly the released checkpoint is materially better than its own released evaluation reports: nRMSE at $h = 368$ falls from 1.3228 under the released pairing to 0.7572 under the causal one, so the released evaluation overstates its own model’s error by 75%.

7.3 No held-out evaluation. Evaluation trajectories are drawn from training data. For the released checkpoint, trained on the entire file, no held-out measurement is possible at all.

7.4 What the spliced windows cost: nothing measurable. We trained a contaminated arm on 7,882 windows — the clean 7,687 plus 195 splices — and, because that confounds *content* with *count*, a duplication control adding the same 195 windows as exact copies of windows already present.

The arm’s contamination rate is 2.47%, against the reference pipeline’s 3.53%. It is deliberately lower: we splice only the 5 boundaries whose *both* sides are training episodes, because 4 of the 9 put held-out rows into training. That is a leakage problem rather than a physics one, and including it would have invalidated our own comparison. So this experiment measures the cost of training on physically impossible transitions, and not the reference’s full exposure.

Training loss over the final 250 iterations: duplication costs 0.90%, splicing costs 21.57%. The bootstrap interval on duplicated — clean is [-0.0061, 0.0215], including zero. So the rise is caused by splice content, not by dataset size — a control we ran only because the first version of this finding inferred the mechanism without it.

In rollout, across 32 cells (two arenas $\times$ two trajectory lengths $\times$ two checkpoints $\times$ two horizons $\times$ two metrics), contamination hurts in 0 of 32 and helps in 9 (Figure 5a). The control is inert, differing from clean in 2 cells.

"Costs nothing" is the wrong summary, and we should not use it. Splicing raises training loss by 21.57% against duplication’s 0.90%, and improves rollout in 9 of 32 cells. Both are measured effects

in opposite directions, which is the signature of regularisation: the spliced windows contain transitions the model cannot fit, it fits the rest less tightly as a result, and it rolls out slightly better. The defensible statement is that **at this rate the splices do not harm rollout and appear to help slightly**, not that they cost nothing. **The unmarked boundaries remain a real defect on leakage grounds; what is now measured is that the physically-impossible-transition component costs nothing detectable at this rate.**

8 The released checkpoint’s variance state is unreachable at the stated iteration counts

The collapse rate is a clock. Fitting it across our runs and extrapolating to the released checkpoint’s σ state implies **153,270** optimisation steps at the configured learning rate. The refit from our 10,000-iteration runs gives 158,319 and 158,003, spreading 3.3% across the three fits — a linear extrapolation validated over a fourfold extension.

The released configuration says 500 iterations. The paper says 2,500. The checkpoint is tagged 5,000. A second, independent parameter on a slower gradient path implies the same order. And under `gaussian_nll` the implied count is *negative*, which identifies the branch the checkpoint was trained with.

What this extrapolation assumes, and what would falsify it. It assumes constant-rate Adam at the configured learning rate from the released initialisation. Five things would break it, and they are not equally plausible:

assumption	if violated	ruled out by the second parameter?
no learning-rate schedule	a decaying schedule inflates the implied count; a warm-up deflates it	partly — <code>min_logstd</code> and <code>log_delta_logstd</code> travel at rates differing by about 5×, and a uniform schedule scales both, so a schedule alone cannot reconcile them without also changing their ratio
<code>log_delta_logstd</code> initialised as released	a different initialisation moves the origin of the fit and rescales the count linearly	no — this is the weakest point of the argument
no warm start from an earlier checkpoint	a warm start makes the count a lower bound on total optimisation, not an estimate of one run	no
no gradient clipping in this path	clipping would slow the collapse and inflate the implied count	partly — the reference does not clip in the world-model path (X-08), so this is ruled out by source rather than by measurement
bound-loss weight as configured	a different weight scales the rate directly	partly — same ratio argument as the schedule

So the defensible claim is narrower than "cannot have come from the released recipe": **no constant-rate run from the released initialisation at the configured learning rate reaches this checkpoint’s variance state in 500, 2,500 or 5,000 iterations.** A warm start or a different initialisation would explain the gap without any inconsistency, and we cannot exclude either.

What the author says. We wrote to the first author on 21 August 2026 asking exactly this. He replied the same day; the exchange is reproduced in full, anonymised, in the supplementary material (SUPPLEMENTARY_CORRESPONDENCE.md): the released `max_iterations`: 500 is "a typo"; his recollection is 5,000 iterations, "as I always did"; he does not recall how the checkpoint was obtained; and — the part that matters most — "the checkpoint was released after a few iterations of the repo than the setup I used for the submission."

That last point reframes this section. The extrapolation above assumes the *released* initialisation and the *released* learning rate. If the repository drifted between the training run and the release, those are not necessarily the values that produced the checkpoint — and a changed `log_delta_logstd` initialisation is precisely the assumption the table above cannot rule out.

So the finding is not that the release is internally inconsistent. It is that **the released artifacts do not reproduce the released checkpoint’s variance state, and the author’s account is that the released repository is not the one that trained it.** That is a documentation gap between a release and a run — common, worth recording, and much less interesting than an inconsistency. We report the arithmetic because it is what let us detect the gap at all, not as a charge against the work.

9 Method

An append-only ledger. Every claim here has a permanent identifier, an evidence class (source, data, run, external, inference) and a status, in `FINDINGS_LEDGER.md` (189 entries). Claims are never edited in place: one that turns out to be wrong is marked superseded, pointed at what replaced it, and kept.

Pre-registration, and one failure of it. Decision rules were committed to git before the data that tested them, with one exception. Figure 4 gives each lead time from commit timestamps. The fifth bar is negative: the duplication-control rule (§7.4) was stated in conversation before the runs but reached git **2.9 hours after they finished**, and we found it only by auditing our own git log. The measurement stands — the arm was built without reference to its outcome — but the claim that it was pre-registered does not, and we withdraw it. A discipline that is only checked when it succeeds is not a discipline.

Six retractions on our own evidence, out of 15 superseded claims kept in the record, plus two that withdraw framings rather than numbers (Appendix D lists them). The most consequential is the second framing retraction: the inference from per-dimension sign counts to a binomial P-value, which assumed an independence the 45 state dimensions do not have (§6.6). Found by our own pre-submission audit, it withdraws the strength of evidence behind what an earlier draft called the strongest result here.

A statistic that was resampling the wrong unit. Our bootstrap pooled three seeds over a shared trajectory set and resampled the pooled vector while reporting the independent-trajectory count, so each trajectory appeared three times. Resampling trajectories instead widens intervals by a mean $1.42\times$ and changes 1 of 16 verdicts, in an $h = 8$ cell already recorded as unresolvable. Every long-horizon verdict survives; both units are reported.

Reproducibility, and a build that checks its own prose. `./reproduce.sh -quick -force` regenerates 29 artifact files and 6,073 numeric values from a clean clone, 6,073 of them bitwise identical (100.00%), 0 differing. Verifying that every numeral came from an artifact says nothing about the sentence built around it — six defects in an earlier draft were of exactly that kind, all downstream of correct numerals. The build therefore also verifies **32 comparative claims** across 15 kinds, each pinning a fragment of the paper’s own text *and* a relation recomputed from the artifacts; all pass, and each is run against a deliberately corrupted expectation on every build and must fail, 31 of 31 caught. **Appendix D gives the argument, the kinds, the self-test, two defects found in the checker itself, and the two exclusions from the numeric comparison.**

10 Actionable lessons

Six things a practitioner can apply without reading the rest of this paper.

Use ensemble disagreement as a ranking signal; it earns its cost. Do not read it as a distance. And expect it to degrade with horizon. At one forecast step it correlates $+0.994$ [$+0.918$, $+0.999$] with realised error — very nearly a perfect ranking. Over the full 20-trajectory rollout it falls to $+0.605$ [$+0.545$, $+0.694$]. That decay is the useful part: the signal is excellent where you can check it cheaply and merely good where you most need it. It still beats the free alternative — the forecast step index — at every horizon we tested, on a paired test that excludes zero at every horizon where the index is defined, and it retains $+0.596$ once that index is partialled out (§6.7). That is a real signal, not a re-encoding of how far ahead you are looking — and not merely a report of which episode is hard: with both the forecast depth and the rollout held constant it still correlates $+0.419$ [$+0.318$, $+0.576$] with error (§6.7). But it is too small to be an interval by a wide margin — $34.4\times$ on the released checkpoint and $13.0\times$ on the ensemble-5 arms we trained — and a risk gate or safety margin that reads σ as a distance is not supported at any horizon, on either.

If you need the interval, rescale per horizon, not globally. One multiplier per forecast horizon, fitted on held-out data, brings coverage within 10 points of nominal on every held-out cell; a single global multiplier manages 2 of them (§6.8). The held-out cells are 6 horizons $\times$ two fold directions on the same 4 trajectories, not independent trials, so read the sweep as consistency and the per-cell deviations as the evidence. The fitted multipliers span $9.31\times$ across horizons, which is precisely why one number cannot serve.

Do not convert per-dimension sign counts into P-values. State dimensions in a robot are physically coupled and share a forecast-depth trend, so an independent-trials null is badly wrong — in our tables by up to $10^{13}\times$ (§6.6). Permute whole trajectories instead. We shipped the binomial version in an earlier draft and it made our weakest evidence look like our strongest.

Count independent trajectories, not trajectories. Two 400-step windows that overlap at all are one piece of evidence, not two. The held-out arena here contains 4 independent 400-step trajectories however many windows are drawn from it, and that number — not the window count — bounds every long-horizon claim. Reporting an interval beside a trajectory count rather than an independent-trajectory count overstates precision, and resampling pooled seed $\times$ trajectory values instead of trajectories narrows intervals by a further $1.42\times$ (§9).

Anchor a decision rule to the horizon the claim is about. Our first pre-registered rule was anchored at $h = 8$, the training forecast horizon, and returned "cannot be settled". The claim was about deployment horizons. The rule was correct in form and pointed at the wrong regime, which is a failure mode that pre-registration does not protect against on its own.

Check that the implemented loss is the described loss before reproducing any number from it. The paper describes two loss terms; the implementation has 7. The predicted variance has an optimum at zero under the implemented one, which is why the released checkpoint's σ is $20,669\times$ smaller than its own error. Reading the loss took an afternoon and explained a result that would otherwise have looked like a training bug.

11 Broader impact

This is a reproduction of a dynamics model on public simulation data, and the reproduction itself carries no significant risk of harm: no new capability, no personal data, no deployment.

The finding does bear on safety, in one specific way worth stating. The method this paper examines uses its uncertainty estimate as a **trust metric** — a reward penalty that steers a policy away from states the model is unsure about. That use is supported by our measurements. But a downstream user who reads the same quantity as a *calibrated interval* — a safety margin, a confidence bound, a gate on when to hand control to a fallback controller — would be materially misled: at the horizon the method itself rolls out over — $h = 100$ — the released checkpoint's ensemble disagreement is $33.4\times$ smaller than the realised error, giving 4.61% coverage where 68.27% is expected. On hardware, a margin that is wrong by that factor is the difference between a conservative controller and one that believes it is safe when it is not.

We think that makes the finding worth publishing rather than the reverse, and it is the reason §6 reports coverage rather than only correlation.

12 Limitations

Effective sample size bounds every long-horizon claim. The out-of-sample arena has 4 independent 400-step trajectories. That is the binding constraint on §5, and no amount of trajectory oversampling changes it.

Ensemble size — no longer an open question, but not a closed one either. Our main experiment runs at ensemble size 1, where the epistemic term is identically zero by construction, so every epistemic measurement here was originally made on the released checkpoint alone. We since trained 3 Arm A arms at ensemble size 5 (§6.7). They reproduce the *direction* of §6.7's finding in 12 of 12 seed-horizon cells and the *calibration* failure at $13.0\times$ — but the pre-registered rule governing the replication returns **DOES NOT GENERALISE**, because its second condition needs the paired difference to exclude zero at a majority of horizons and it does so at 1 of 4. The binding constraint is the same one this section opens with: our arms have a genuine held-out arena of only 4 independent trajectories, and the rule was written without checking what it could detect there. **So §6.7's finding is established on the released checkpoint and supported but not established on a model we trained.**

One dataset, one gait, one terrain. All commands are drawn from one bounded box and the gait is a single trot throughout. "Generalisation" here means across velocity commands, not across gaits or terrain.

The per-horizon recalibration is fitted and tested on two episodes only. §6.8's remedy transfers across the two held-out episodes in both directions, which is the strongest test the released split allows, but

two episodes is not a demonstration that the multipliers transfer to a new robot, gait or terrain. Treat the lookup table as a recipe to refit, not as constants to copy.

Two secondary analyses rest on a single training seed — the long-horizon trend fit and the per-dimension matched comparison, both computed on seed 1 alone. The headline A/B result is not among them: it is a three-seed mean with per-seed values reported (§5). This is recorded in the artifacts themselves.

We did not measure what the miscalibration costs. We show that the penalty the follow-up applies is miscalibrated as a scale — $33.4\times$ overconfident at $h = 100$, the horizon its own imagination rollouts run to — but the only use the method makes of that quantity is to shape policy learning, and we did not train a policy. A miscalibrated scale that enters as a relative penalty across candidate actions may cost little, or may cost a great deal; our measurements cannot distinguish those. **The finding bounds what the quantity reports, not what it costs.** That distinction is easy to lose and we do not want a reader to take the ratio as a measure of harm.

The per-dimension ordering tests are underpowered at every sample size we can reach. Once the coupling between state dimensions is respected (§6.6), the out-of-sample arena’s 4 independent trajectories admit a smallest attainable P-value of 0.04167 — coarser than the multiplicity-corrected threshold 0.002, so that arena cannot reject at any effect size whatever. The larger arenas can reject and do not: over all ten episodes the smallest P in the family is 0.0042 against a threshold of 0.002. Resolving this needs more episodes than the released dataset contains, not a better test. Note the scope: this limits the *per-dimension* evidence. The aggregate scalar the method applies is separately and more strongly supported (§6.7), on the same trajectories, because it is one test rather than forty-five coupled ones.

We did not reproduce the policy-learning results of either paper. This is a dynamics-model reproduction only.

13 Conclusion

The Robotic World Model’s central training claim reproduces, and the margin is large. Neither uncertainty output of the follow-up that adds them reports what a reader would take it to report. The aleatoric σ is $20,669\times$ smaller than its own error, and the cause is that the objective’s optimum is $\sigma = 0$ with the term that should prevent this cancelling out of the gradient. The epistemic term the method actually penalises with is $600\times$ better and still $34.4\times$ overconfident where it is used.

The more useful finding is asymmetric, and it cuts both ways. The scale failure is established and large — but it is repairable: a per-horizon multiplier, fitted on one held-out episode and scored on another, restores nominal coverage on every held-out cell where a global multiplier restores 2 of them — 6 horizons in each of two fold directions, on the same 4 trajectories, so not independent trials. And the ranking use the follow-up claims does survive a real test: against the forecast step index, a free baseline neither original paper ran, ensemble disagreement wins at every horizon, keeps $+0.596$ once the index is partialled out, and still reaches $+0.739$ when depth is held exactly constant — so it is not a re-encoding of the clock. That is the one claim of either original work that this reproduction strengthens rather than qualifies.

What does not survive is the per-dimension form of the ordering evidence. Three of the five σ estimates we measured order their own errors better than chance in direction — the epistemic term on every one of the 45 dimensions at $h=368$, and the faithful and teacher-forced arms. That count is a direction, not a tally of independent trials — the dimensions are physically coupled, and the permutation test over whole trajectories is the statistic (§6.6). The released checkpoint’s *aleatoric* head does the opposite, ranking error inversely on every one of 45 dimensions over all ten episodes and at chance on the held-out pair alone — a dependence on arena that §6.6 sets out. The corrected arm sits at chance in both. And once the physical coupling between state dimensions is respected by permuting whole trajectories, no per-dimension count in this paper reaches significance after multiplicity correction. We report that rather than the independent-trials P-values an earlier draft carried, which were wrong by up to a factor of about 10^{13} on the cells we had cited as evidence. Neither quantity yields a usable interval. Uncertainty in this family of models should be read as a weak ordering at best, or fixed at the objective; it should not be read as a scale, and a ranking use deserves its own validation on the deployment distribution rather than trust inherited from here.

14 Data and code

The full repository — code, every artifact under `results/`, and `FINDINGS_LEDGER.md` with the complete claim record including the retractions — accompanies this submission as anonymised supplementary material, and will be released under a permanent archival identifier on acceptance. Neither upstream repository is redistributed; `setup.sh` fetches both at pinned commits and verifies two SHA-256 hashes.

The pre-registration argument in §9 rests on commit timestamps, and those are author-settable via `git commit -date`. That matters, because §9 is load-bearing. Two things address it. The supplementary material includes an anonymised `git log` covering every commit cited here, so the ordering in Figure 4 is checkable at review time. And **the repository was archived by Software Heritage on 21 August 2026**, before submission, under a permanent identifier whose visit timestamp is not author-controllable; the identifier resolves to a named repository and is therefore disclosed on acceptance rather than here.

What that archive establishes should be stated precisely, because it is easy to overclaim. It does **not** prove any individual commit date is genuine. It proves that the repository, with the whole pre-registration history in the form this paper cites, existed no later than that archival moment, as recorded by a third party with no interest in the claim — so nothing in the record can have been back-dated afterwards. That bounds §9 rather than proving it, and a reviewer should read it as such.

15 References

1. C. Li, A. Krause, M. Hutter. *Robotic World Model: A Neural Network Simulator for Robust Policy Optimization in Robotics*. arXiv:2501.10100v1, 17 January 2025. Read at v1, whose Roman-numeral sectioning our references follow; v2 (23 April 2025) renumbered to Arabic and moved IV-C into Appendix A.4.1.
2. C. Li, A. Krause, M. Hutter. *Uncertainty-Aware Robotic World Model Makes Offline Model-Based Reinforcement Learning Work on Real Robots*. arXiv:2504.16680v1, 23 April 2025. Read at v1; now at v3, last revised 8 Jan 2026. §5.1 and Eq. 4–5 keep their numbers there; every figure and appendix table has moved, and the model is renamed RWM-O to RWM-U. The crosswalk is in `results/original_paper_figures.json`.
3. Z. Abbas, S. Sokota, E. J. Talvitie, M. White. *Selective Dyna-style Planning Under Limited Model Capacity*. ICML 2020. arXiv:2007.02418.
4. K. Chua, R. Calandra, R. McAllister, S. Levine. *Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models*. NeurIPS 2018. arXiv:1805.12114.
5. C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger. *On Calibration of Modern Neural Networks*. ICML 2017. arXiv:1706.04599.
6. M. Janner, J. Fu, M. Zhang, S. Levine. *When to Trust Your Model: Model-Based Policy Optimization*. NeurIPS 2019. arXiv:1906.08253.
7. R. Kidambi, A. Rajeswaran, P. Netrapalli, T. Joachims. *MOReL : Model-Based Offline Reinforcement Learning*. NeurIPS 2020. arXiv:2005.05951.
8. V. Kuleshov, N. Fenner, S. Ermon. *Accurate Uncertainties for Deep Learning Using Calibrated Regression*. ICML 2018. arXiv:1807.00263.
9. B. Lakshminarayanan, A. Pritzel, C. Blundell. *Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles*. NeurIPS 2017. arXiv:1612.01474.
10. C. Lu, P. J. Ball, J. Parker-Holder, M. A. Osborne, S. J. Roberts. *Revisiting Design Choices in Offline Model-Based Reinforcement Learning*. ICLR 2022 (Spotlight). arXiv:2110.04135.

11. Y. Ovadia, E. Fertig, J. Ren, Z. Nado, D. Sculley, S. Nowozin, J. V. Dillon, B. Lakshminarayanan, J. Snoek. *Can You Trust Your Model’s Uncertainty? Evaluating Predictive Uncertainty Under Dataset Shift*. NeurIPS 2019. arXiv:1906.02530.
12. T. Yu, G. Thomas, L. Yu, S. Ermon, J. Zou, S. Levine, C. Finn, T. Ma. *MOPO: Model-based Offline Policy Optimization*. NeurIPS 2020. arXiv:2005.13239.

Entries 3–12 are the §2 bibliography. Each was checked against the paper itself: title, full author list and venue from the arXiv record, and for any sentence this paper attributes, the sentence matched verbatim against that paper’s own text — 10 of 10 entries and 9 of 9 attributed fragments (*results/t1_bibliography_verified.json*).

A Verification chain

What every downstream number rests on. Each level was passed before the next was attempted.

level	claim	result
shapes	parameter counts match the reference	exact
wiring	inference outputs match the reference module	0.000e+00 , bitwise
indexing	the harness feeds the actions it claims	bitwise against the raw CSV
residual	the zero-delta model is the hold-last floor	1.192e-07
objective	losses and gradients match	0.000e+00 across 7 terms, 106 tensors
trainer	can memorise a single batch	1,506× loss reduction

B Reproducing

```
./setup.sh # clone upstreams at pinned commits
python3.11 -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt
./reproduce.sh --quick --force # everything except training
```

`-force` matters: a clean clone already contains each stage’s declared output, so without it every stage skips.

Runtime. Training stages are excluded by `-quick`, which is what makes the quick path practical. Training all 24 runs takes **45 hours** of recorded wall clock on two CPU cores: 20 hours for the 6 runs at 10,000 iterations and 25 for the remaining 18 at 2,500. The longest single run is 4.4 hours. An earlier version of this appendix said 22 hours; that figure predated the 6 ten-thousand-iteration runs added for the three-seed headline, and is corrected here from the `wall_clock_s` field of every run artifact rather than re-estimated.

C Figures

D Verifying the paper’s own claims

The six numbered retractions, in order. In order: a premise about forecast decay that turned out not to exist in the code; a framing of the released checkpoint as "clearly informative" that rested on an $n=10$ estimate we ourselves showed to be biased low; an aggregation artifact that inverted a published-model comparison in our favour, withdrawn when the gating checks we had written refuted it; a per-dimension comparison that turned out to be unmatched; the claim that σ is input-independent "in all four models", made against a table holding three; and the phrase "the released checkpoint’s uncertainty output", singular, when the checkpoint emits two and we had measured the one the method discards. The two framing retractions are the claim that a pre-registration was pre-registered, and the binomial inference of §6.6. Each is a numbered entry in `FINDINGS_LEDGER.md` with its evidence and its successor.

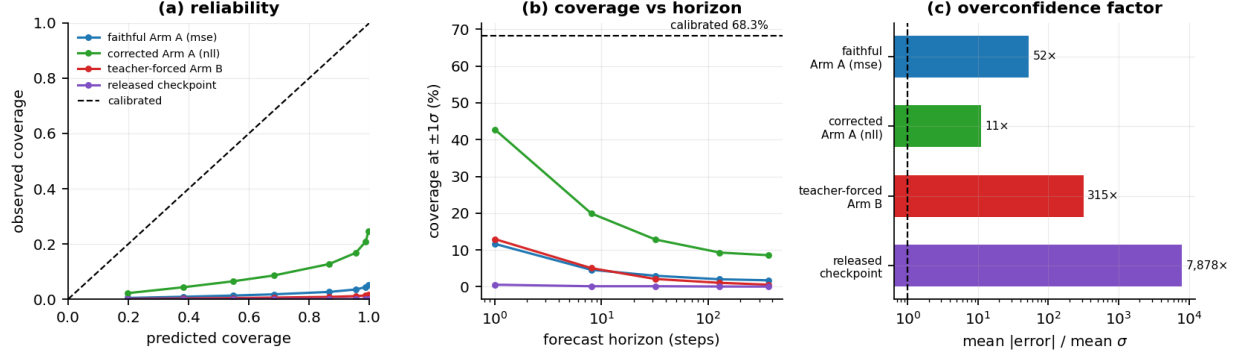


Figure 1: Calibration of all four models on the held-out arena. (a) reliability: observed against predicted coverage, with the calibrated diagonal. (b) coverage at $\pm 1\sigma$ against forecast horizon, log scale, against the 68.3% a calibrated Gaussian gives. Every curve sits far below the diagonal and falls further with horizon.

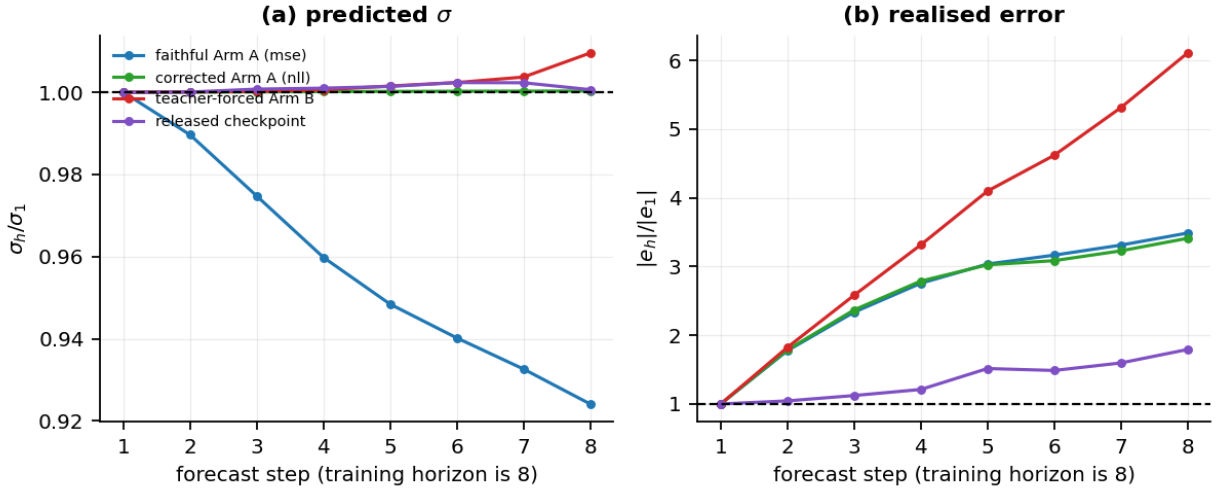


Figure 2: Why the coverage collapse is a horizon effect. Both panels are normalised to forecast step 1. (a) predicted σ barely moves, and for the faithful arm it declines. (b) realised error grows by an order of magnitude over the same steps. The gap between the panels is the collapse.

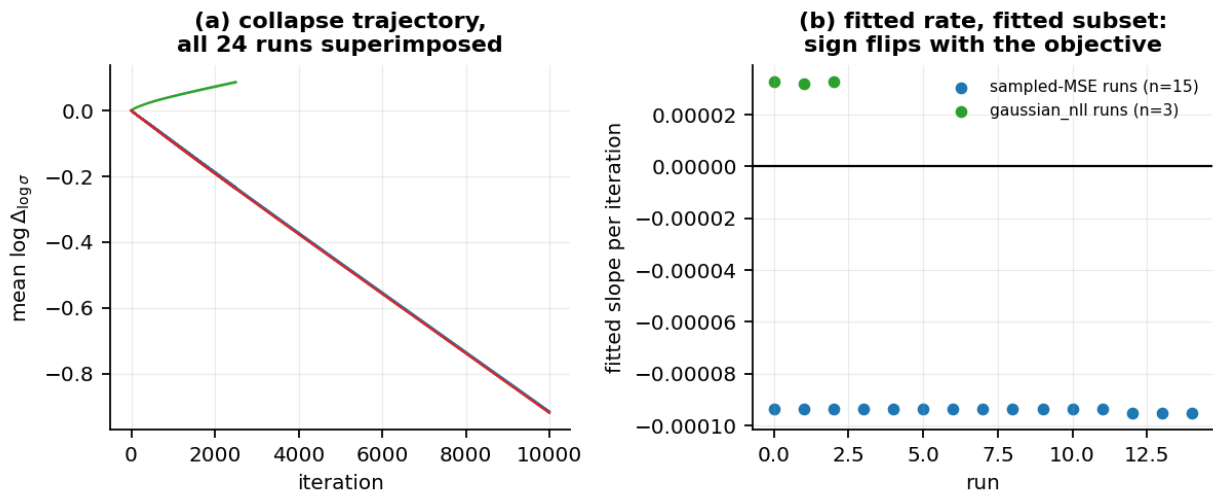


Figure 3: The variance collapse is objective-driven. (a) mean $\log \Delta_{\log \sigma}$ against training iteration for every run. (b) the fitted per-iteration slope for each run, grouped by objective: negative and tightly clustered under sampled MSE, positive under `gaussian_nll`. The sign flip is the evidence that the objective, not the optimiser or the data, produces it.

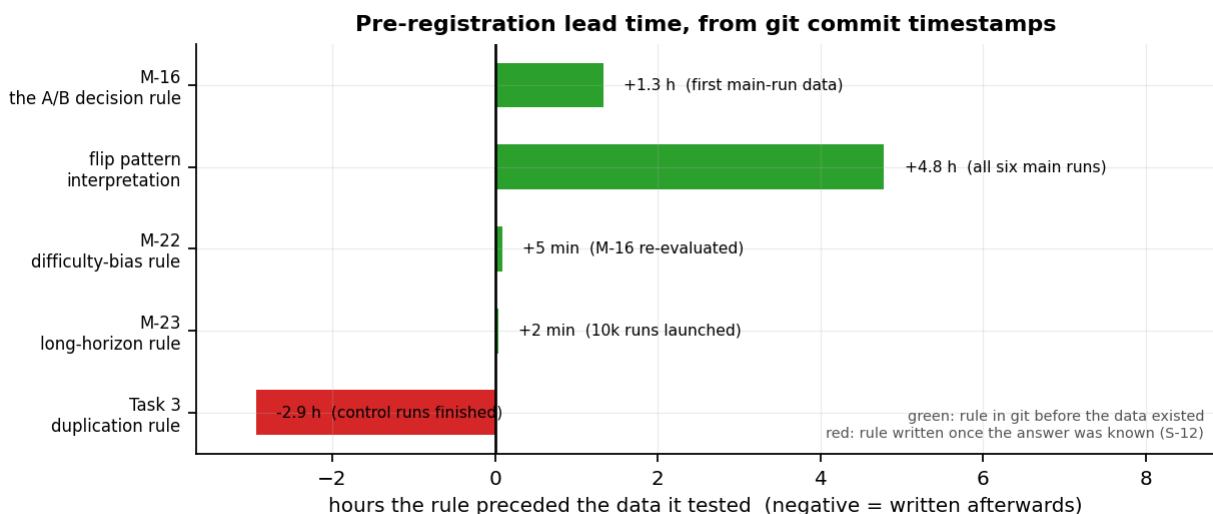


Figure 4: Pre-registration lead time for each decision rule, from git commit timestamps. Positive is a rule committed before the data that tested it existed; negative is a rule written afterwards. The one negative bar is the Task 3 duplication rule, retracted as a pre-registration in this paper.

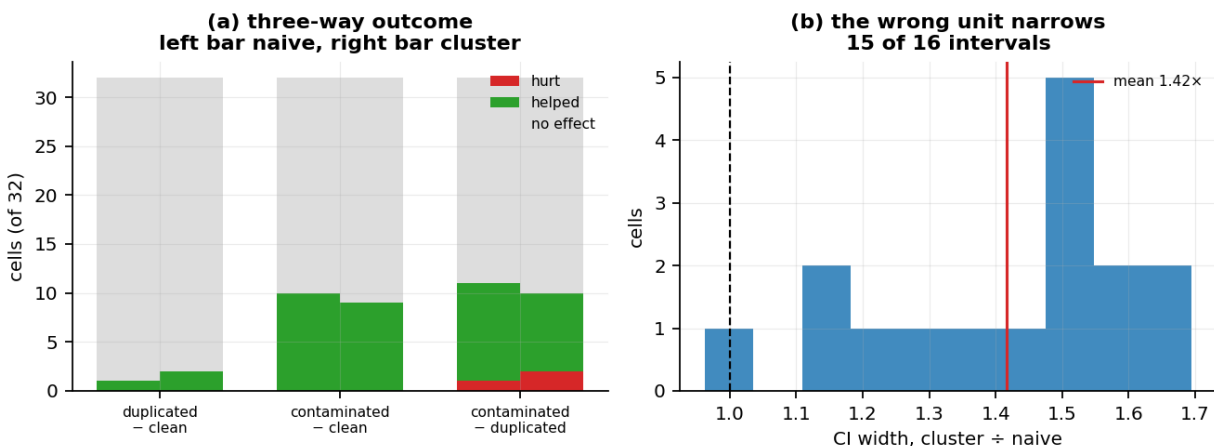


Figure 5: The contamination control. (a) outcome across 32 cells for each arm pair, naive bootstrap on the left of each position and cluster bootstrap on the right; the duplication control is inert. (b) distribution of the ratio of cluster to naive confidence-interval width, with the mean marked. Resampling trajectory-step pairs rather than whole trajectories narrows every interval.

`build_paper.py` asserts that every printed number came from a named artifact. That is a guarantee about *provenance*, and it is silent about *relations between* provenanced numbers. Five failure modes survive it, and all five occurred in this paper:

- **an interval relation that is not the one asserted** — "the intervals do not overlap", where at $h=128$ they overlap across 0.604–0.643;
- **an extremum that is not the extremum** — the worst-calibrated held-out cell named as epistemic at $h=1$, which is third; the largest deviation is aleatoric at $h=128$;
- **a stated change with the wrong sign** — "a change of +0.010", where partialling the forecast index out *reduces* the correlation;
- **two prose descriptions of one ratio that disagree** — "nearly three orders of magnitude" in the abstract against "two orders" in §13, of 600×
- **a count attributed to the wrong evaluation arena** — 0 of 45 over all ten episodes asserted where the table beside it printed the held-out arena's 20 of 45.

None is a numeral. None appears in `results/paper_numbers.json`. Each was typed.

The check kinds. `scripts/check_comparative_claims.py` verifies 32 claims across 15 kinds: *overlap* (two intervals do or do not overlap), *extremum* (a named cell is the max or min of its family), *sign* (a stated rise or fall matches the direction of the difference), *orders* (a stated count of orders of magnitude matches $\text{round}(\log_{10}(\text{ratio}))$), *cell* (a k-of-45 count is the arena and horizon the text names), *compare* (a stated ordering between two scalars), *relvar* (a stated ratio of relative variabilities), and *count-consistency* (one count asserted in several places, in words or numerals, agrees with the ledger everywhere).

Each entry pins two things and requires both: a **fragment of the paper's own text**, so that rewording a sentence fails the check rather than silently detaching it from the claim it guards, and a **relation recomputed from the artifacts**. A check that only re-asserts an artifact fact guards nothing; a check that only matches text guards nothing either.

The self-test. Every assertion is run against a deliberately corrupted expectation on each build and must fail: the interval relation inverted, the extremum replaced by the *runner-up* rather than an absent label,

the sign flipped, the order of magnitude and the dimension counts moved by one. 31 of 31 are caught. An assertion that has quietly stopped being able to fail is worth less than no assertion, because it reads as coverage.

Two defects the self-test found in the checker itself. Its first version applied a fixed corruption per kind — `expect: "disjoint"` to every overlap check — so for claims that already expected that value the corruption was a no-op, and two of eleven assertions reported as missed. They were not missed; nothing had been corrupted. Corruptions now invert relative to each claim’s own expectation. Later, a label helper prefixed a horizon to family keys that were already model names, producing `h=teacher-forced armB`, which matched nothing and failed two checks whose extrema were correct. Both were defects in the checker rather than in the paper, and both surfaced because the checks were run rather than assumed.

Two exclusions from the numeric comparison, on the same principle in both cases: the number measures the machine, not the model.

The CPU budget. 4,282 timing fields and the 586 values of `results/step4_5_timing.json` — projected runtimes for configurations we did not run, peak resident memory, and the standard deviation across repeats. It cannot reproduce bitwise on another machine, or on this one under different load, and it records that about itself: across its 4 configurations the standard deviation of seconds-per-iteration across repeats runs from 5% to 32% of the mean (`ens1_bs256`) — on one machine, within a single measurement session.

One wall-clock-bounded diagnostic. `results/step4_4_overfit_ens1.json` stops after 2,700 seconds rather than at its 2,000-iteration cap, so it reaches a different iteration count on every machine — three different values across the three hosts we have run it on. Its iteration count and terminal losses are therefore a property of the host, and we do not quote any of them here: a number the build declares host-dependent has no business being printed as a result. **Its sibling from the same script is not excluded:** that run reaches its cap, and reproduces bitwise. Excluding by filename rather than by stopping rule would have dropped the reproducible one along with it, so the verifier decides from the artifact — a run that stopped short of its own cap was time-bounded.

Excluding a file is not sufficient on its own. `results/paper_numbers.json` records the *source* of every value it holds, and it had copied that diagnostic’s iteration count into a key of its own — so the host-dependence leaked through a file that was not excluded, and the clean clone duly differed on it. The verifier now drops any key whose recorded source is an excluded artifact (7 of them), which follows the provenance the file already carries rather than requiring anyone to remember.

E What testing the untested claims would require

§4’s table marks 4 claims tested and the rest not. "Not tested" is an apology unless it comes with a price, so here is what each would cost. We give compute orders where we can estimate them honestly from this project’s own measurements and say so where we cannot.

Everything below needs what this reproduction did not have: a simulator. Our arms train on the released CSV, which is a recording. Every untested claim needs *interaction* — a policy acting in an environment and the environment responding — and that means Isaac Lab, which needs an RTX-class NVIDIA GPU. No amount of CPU substitutes: the reference’s data generation is GPU-parallel simulation, not a data-loading problem.

untested claim	what it needs	order
Sample efficiency, 6,000,000 against ~20M transitions (§IV-E)	Isaac Lab, an RTX-class GPU, the MBPO-PPO loop, and a PPO baseline run to convergence for the comparison	the reference reports 6,000,000 pretraining transitions and 50 min of 20M training on their hardware; the PPO baseline’s 20M is the dominant cost
MBPO-PPO beats SHAC and Dreamer (§IV-E)	the above, plus SHAC and Dreamer implementation at matched budgets	three policy-learning stacks, each tuned enough that the comparison is fair — the largest engineering item here
Zero-shot hardware transfer (§IV-E)	all of the above, plus an ANYmal, a safe test area, and the sim-to-real stack	not estimable in compute; the binding constraint is hardware access, not GPU hours
Whether the penalty improves the learned policy (250k.16680v1 §5)	Isaac Lab, the MBPO-PPO loop, and at minimum an ablator with the penalty weight at zero	one policy-learning stack; the cheapest of the four, and the one that would bound §12’s open question about what the mislocalization costs
Beats MLP, RSSM, transformer baselines (§IV-D)	no simulator needed — but the lite release ships only the RNN variant, so all three baselines would have to be implemented	comparable to our own model’s 45 h of CPU training per architecture, times three, if run at our data budget
M=32, N=8 optimal (§IV-C)	no simulator needed; a sweep over M and N at our data budget	our 24 runs took 45 h on two cores; a modest sweep is a small multiple of that

The two at the bottom are within reach of this setup and are the honest next steps for anyone extending this work on CPU. The four above them are not, and no amount of care with the released CSV changes that.

